

Ψηφιακή Επεξεργασία Εικονας

Εργασία 3

Σταματιος Ρουσος

July 2025

1 ΕΙΣΑΓΩΓΗ

Λοιπον, δε θα πω πολλα, γιατι εχει και πολλη ζεστη. Η ολη υποθεση ειναι:

Φορτωση εικονας $\rightarrow$ Μετατροπη σε Γραφο $\rightarrow$ Υπολογιζω βαρη και δημιουργω affinity matrix $W \rightarrow$ Βρισκω Laplacian Matrix $L = D - W \rightarrow$ Λυνω προβλημα ιδιοτιμων αναλογα με τη μεθοδο $\rightarrow$ Παιρνω τις k μικροτερες ιδιοτιμες που προκυπτουν και τα ιδιοδιανυσματα που αντιστοιχουν σε αυτες $\rightarrow$ Βαζω τα ιδιοδιανυσματα σε ενα πινακα $U \rightarrow$ Ομαδοποιω τα σημεια του U σε clusters-κατηγοριες $C_1, C_2, \dots, C_k$ και τα εμφανιζω.

Στην ουσηα παιρνω την αρχικη εικονα και κατατμιζω τα pixels σε clusters - δηλαδη κατηγοριες χρωματων. Επειτα ομαδοποιω τα pixels. Επισης εκτιμω ποσο καλος ηταν ο διαχωρισμος. Περισσοτερα λογια για το πως γινεται καθετι, θα πω στα αντιστοιχα αρχεια που επιτελουν τις παραπανω λειτουργιες.

Παμε να δουμε τι κανει καθε αρχειο και τα πειραματικα αποτελεσματα που δινει.

2 Image to graph.py

Μετατρέπει την εικόνα εισόδου σε γράφο. Κάθε pixel, που είναι ένα διάνυσμα της μορφής $\mathbf{p} = [R \ G \ B]$, γίνεται κομβός και όλα τα ζεύγη κομβών ενώνονται μεταξύ τους με ακμές. Κάθε ακμή έχει ένα βάρος $A(i, j) = \frac{1}{\exp(d(i, j))}$, όπου $d(i, j)$ η Ευκλείδεια απόσταση της φωτεινότητας μεταξύ του i -οστού και j -οστού pixel, δηλαδή το μέτρο της διαφοράς των διανυσμάτων των 2 pixel:

$$d(i, j) = \sqrt{(R_i - R_j)^2 + (G_i - G_j)^2 + (B_i - B_j)^2}$$

Τα βάρη A των ακμών δείχνουν ποσο "ισχυρή" είναι η κάθε ακμή, δηλαδή ποσο μοιάζει το ένα pixel με το άλλο. Όσο περισσότερη χρωματική ομοιότητα έχουν, τόσο μεγαλύτερο το βάρος A της ακμής που τα ενώνει.

Ο **affinity matrix** είναι ο πίνακας που ως στοιχεία έχει ακριβώς αυτά τα βάρη A , δηλαδή την ομοιότητα των pixels.

3 Spectral Clustering.py

Λοιπόν τι κάνει αυτό το αρχείο τώρα. Κάνει καταταμηση εικόνας. Πώς;; Παιρνει τον **affinity matrix** $\mathbf{W}$ που φτιαξαμε πριν με τα βάρη των ακμών, και υπολογίζει τον **Laplacian Matrix** $\mathbf{L}$ ως:

$$\mathbf{L} = \mathbf{D} - \mathbf{W}$$

Οπου **D** ο διαγωνιος πινακας ο οποιος οριζεται ως:

$$D(i, i) = \sum_j W(i, j)$$

Κατοπιν λυνει το προβλημα ιδιοτιμων:

$$Lx = \lambda x$$

και παιρνει τις **k** μικροτερες ιδιοτιμες και τα ιδιοδιανυσματα που αντιστοιχουν σε αυτες. Σχηματιζει τον πινακα **U** ο οποιος εχει ως στηλες τα ιδιοδιανυσματα που υπολογισε παραπανω. Ομαδοποιει τις γραμμες του πινακα **U** με τον αλγοριθμο **kmeans**, ο οποιος στην ουσια δειχνει σε ποιο cluster/ομαδα ανηκει το καθε pixel, επιστρεφοντας ενα διανυσμα **labels** μηκους n (οσα και τα pixels). Καθε στοιχειο του διανυσματος **labels** εχει μια τιμη που δειχνει τον αριθμο της ομαδας/cluster που ανηκει το pixel. Αποθηκευει τα αποτελεσματα σε ενα πινακα **cluster_idx**.

4 Demo1.py

Το demo1 απλα λαμβανει τον δοσμενο **affinity matrix d1a** και εφαρμοζει τη μεθοδο spectral clustering πανω του για **k = 2, 3, 4**.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
PS C:\Users\STAMATIS\Documents\Erg_3\file
Spectral clustering for k = 2:
[1. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0. 0.]

Spectral clustering for k = 3:
[1. 1. 1. 1. 0. 0. 0. 0. 2. 2. 2. 2.]

Spectral clustering for k = 4:
[2. 1. 1. 2. 0. 0. 0. 0. 3. 3. 3. 3.]
```

Figure 1: Αποτελέσματα clustering για $k=2,3,4$

Παρατηρούμε ότι η εικόνα d1a αποτελείται από 12 pixels, και με βάση την τιμή του k , η μέθοδος spectral clustering ομαδοποιεί κάθε pixel της σε k κατηγορίες χρωμάτων. Γι' αυτό σε κάθε λίστα βλέπουμε πληθος διαφορετικών τιμών ίσο με k . ($k=2 \rightarrow 0$ και 1, $k=3 \rightarrow 0,1,2$ κλπ)

5 Demo2.py

Εδώ εφαρμόζουμε πάλι τη μέθοδο **spectral clustering** για $k = 2, 3, 4$ απλά για τις 2 RGB εικόνες εισόδου d2a,d2b. Επομένως, μετατρέπουμε κάθε εικόνα σε γράφο και κάνουμε κατατμήση.

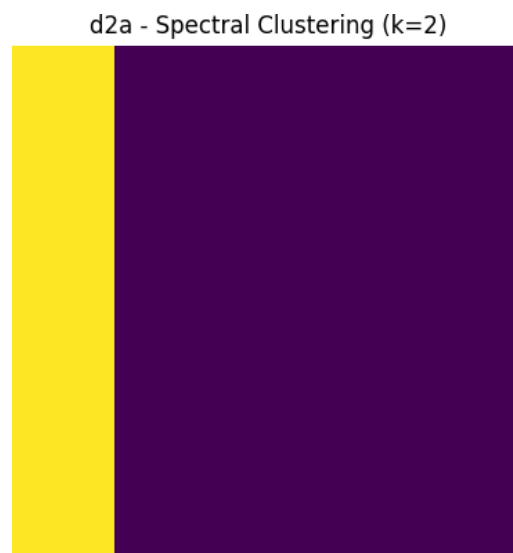


Figure 2: Εικόνα d2a - Spectral Clustering για k=2



Figure 3: Εικόνα d2a - Spectral Clustering για k=3

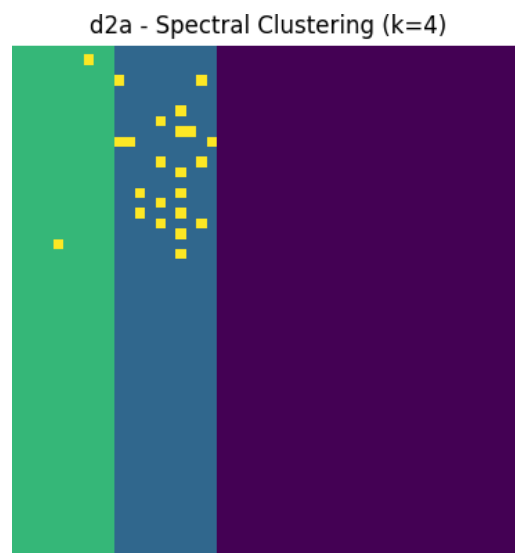


Figure 4: Εικόνα d2a - Spectral Clustering για $k=4$

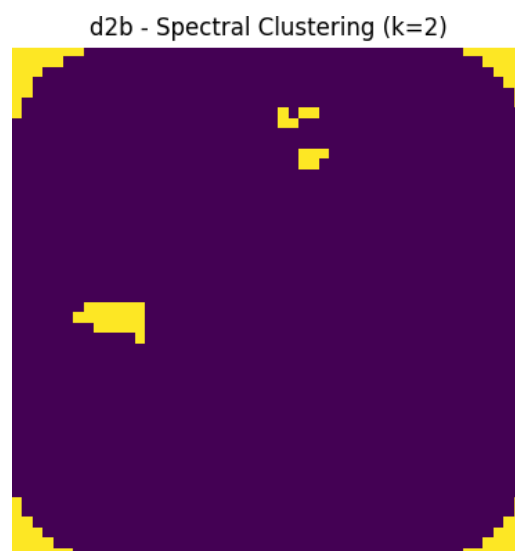


Figure 5: Εικόνα d2b - Spectral Clustering για $k=2$



Figure 6: Εικόνα d2b - Spectral Clustering για $k=3$



Figure 7: Εικόνα d2b - Spectral Clustering για $k=4$

6 Normalized-cuts

Λοιπον τα βασικα κι εδω. Η κατατμηση εικονας γινεται με ιδιο τροπο με πριν απλα τωρα αλλαζει το προβλημα ιδιοτιμων και γινεται:

$$Lx = \lambda Dx$$

Λαμβανει υποψην δηλαδη και τον διαγωνιο πινακα **D**

Λιγα λογια μονο για καθε συναρτηση:

n_cuts:

Εκτελει κατατμηση εικονας ακριβως οπως η spectral, αλλα με διαφορετικο ιδιοπροβλημα. Συμπεριλαμβανει και τον διαγωνιο πινακα **D**. Καθε στοιχειο **D_{ii}** του πινακα **D**, ειναι το αθροισμα των βαρων ολων των ακμων που φευγουν απο τον κομβο **i**. Δηλαδη δειχνει το ποσο "συνδεδεμενος" ειναι ο κομβος στον γραφο. Επομενως, η μεθοδος ncuts λαμβανει υποψην ποσο καλα συνδεδεμενος ειναι ο καθε κομβος. Αν π.χ η εικονα εχει μια μικρη περιοχη εντονου πρασινου, η μεθοδος ncuts θα τη συμπεριλαβει στο σπασιμο της και θα την ξεχωρισει σαν ομαδα. Ενω η spectral, κοιταει πιο πολυ το πληθος των ομοιων κομβων στο σπασιμο της οποτε μπορει να την αγνοησει, γιατι δεν αποτελειται απο πολλα pixels.

calculate_n_cut_value:

Ειναι μια μετρικη που δειχνει το ποσο καλο ηταν το κοψιμο. Οσο πιο μεγαλη τιμη εχει, τοσο πιο "ακριβο"/κακο ηταν το κοψιμο. Οριζεται ως:

$$Ncut(A, B) = 2 - Nassoc(A, B)$$

οπου

$$Nassoc(A, B) = \frac{assoc(A, A)}{assoc(A, V)} + \frac{assoc(B, B)}{assoc(B, V)}$$

και

$$\mathit{assoc}(A, V) = \sum_{u \in A, t \in V} W(u, t)$$

Τι γίνεται τώρα εδώ: Στοχος της ncuts είναι να κάνει τετοιους διαχωρισμους σε ομάδες, που το Ncut value να είναι μικρο. Παμε να δουμε γιατί. Το $\mathbf{N}_{assoc}(A, B)$, όπου A, B δυο ομάδες, είναι ένα αθροισμα 2 κλασμάτων. Τα κλάσματα αυτά είναι:

Αθροισμα βαρων μεταξυ κομβων ιδιας ομάδας A/B

Αθροισμα βαρων κομβων της ομάδας προς ολους τους κομβους V

Αυτά τα κλάσματα είναι μικροτερα της μοναδας και δειχνουν ποσο πολυ συνδεδεμενοι είναι οι κομβοι μιας ομάδας μεταξυ τους, σε σχεση με τις συνδεσεις τους σε ολο τον κομβο. Στην ουσια όσο πιο μεγαλα είναι αυτά, τόσο πιο πολυ/ισχυρα συνδεονται οι κομβοι μιας ομάδας μεταξυ τους παρα με αλλη ομάδα, και ετσι το Nassoc είναι μεγαλο και αρα το Ncut μικρο. Δηλαδη το σπασιμο ηταν καλο.

n_cuts_recursive:

Λοιπον η αναδρομικη τώρα, είναι υποπεριπτωση της ncuts για $k = 2$. Πως δουλενει:

Καλει μια φορα την μη-αναδρομικη ncuts για $k=2$ και σπαι τον γραφο σε 2 κομματια/ομάδες. Μετα, με βαση καποια κριτηρια, ελεγχει αν πρεπει να σπασει καθε κομματι ξανα σε 2 κομματια. Παμε να δουμε τι κριτηρια είναι αυτά. Χρησιμοποιει δυο κατωφλια $T1$ και $T2$. Και λεει: αν καποιο απο τα 2 κομματια που εκοψες (στο 1ο κοψιμο) εχει πληθος κομβων μικροτερο απο $T1$, ή αν η τιμη $\mathbf{Ncut}(A, B)$ είναι μεγαλυτερη απο $T2$, τοτε σταματα τη διχοτομηση για εκεινο το κομματι, και ελεγξε το ιδιο για το αλλο. Αν ολες οι συνθηκες καλυπτονται, ξανακοψε το κομματι στα 2. Καν'το μεχρι να μην μπορει κανενα κομματι να ξανακοπει. Οπως ειπαμε, η μετρικη $\mathbf{Ncut}(A, B)$ είναι μετρο της ποιτητας του κοψιματος. Επομενως η συγκριση

της με το κατωφλι T2, μας λειπει αν ειναι καλη ή οχι η διχοτομηση. Η παραμετρος T1 υπαρχει γιατι ακομα και καλη να ειναι η διχοτομηση, μπορει το σπασιμο μου να ειναι τετοιο που καποια ομαδα, να περιεχει πολυ λιγους κομβους (πχ 2-3 pixels σε εικονα 1000x1000) και αμα ξανασπασει, θα δωσει ασημαντες μικρες ομαδες. Αμα υπηρχε μονο η συγκριση με το Ncut μπορει να μας ξεγελουσε σε τετοια περιπτωση και να ξαναεκοβε την ομαδα με τους πολυ λιγους κομβους. Στην ουσια το T2 λειπει "ειναι θεωρητικα καλο το κοψιμο;;", ενω το T1 λειπει "ειναι αρκετα μεγαλο το cluster που προεκυψε για να αξιζει να συνεχισω;;". Για παραδειγμα, αν εχω μια εικονα με πολυ κοκκινο, πολυ μπλε, και λιγο αλλα αρκετα εντονο πρασινο, και τη διχοτομησω σε 3 clusters, τοτε το Ncut θα ειναι χαμηλο γιατι καθε ομαδα ειναι συμπαγης (μεγαλο $\text{assoc}(A,A)$ σε σχεση με το $\text{assoc}(A,V)$) αλλα δεν εχει νοημα να ξανασπασω την πρασινη ομαδα.

7 demo3a.py

Demo 3 και τελειωσαμε. Το demo3a ζηται να εφαρμοσουμε τη μεθοδο nCuts στις 2 εικονες εισοδου για clusters $k = 2, 3, 4$. Παρακατω τα πειραματικα αποτελεσματα:

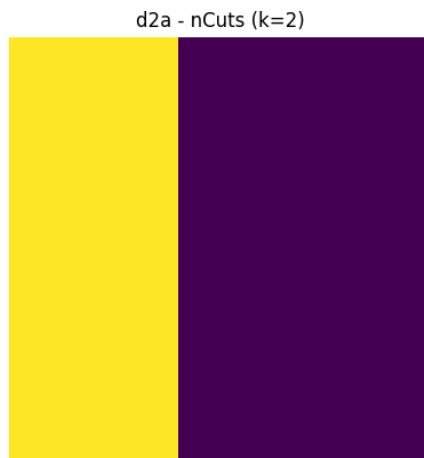


Figure 8: Εικόνα d2a - nCuts για $k = 2$

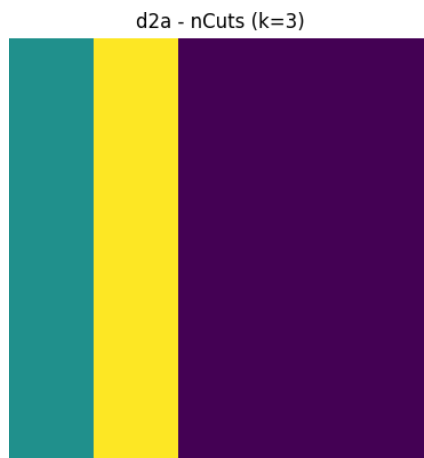


Figure 9: Εικόνα d2a - nCuts για $k = 3$

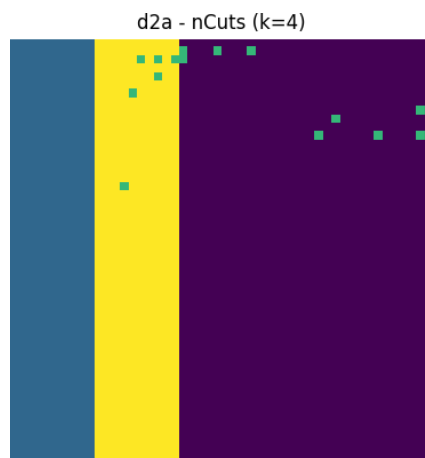


Figure 10: Εικόνα d2a - nCuts για $k = 4$

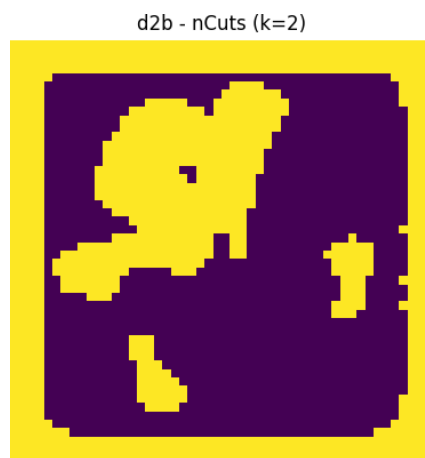


Figure 11: Εικόνα d2b - nCuts για $k = 2$

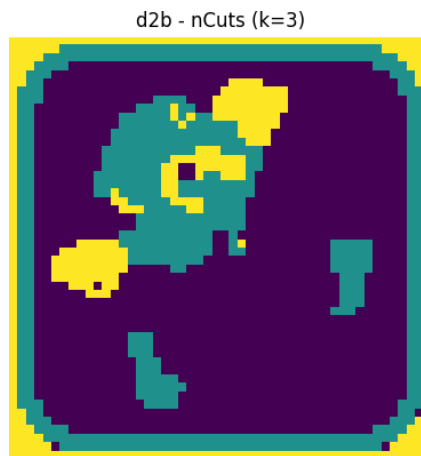


Figure 12: Εικόνα d2b - nCuts για $k = 3$

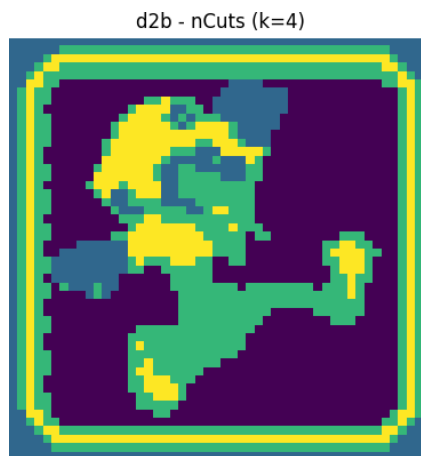


Figure 13: Εικόνα d2b - nCuts για $k = 4$

Παρατηρούμε ότι η nCuts κάνει πολύ καλύτερα, πιο ισορροπημένα κοψίματα, ακριβώς επειδή, όπως εξηγήσαμε, λαμβάνει υπόψη τον διαγωνίο πίνακα D , που δείχνει ποσο πολύ συνδεδεμένος είναι κάθε κόμβος με τους άλλους.

8 demo3b.py

Εδώ θέλει την αναδρομική για κάθε εικόνα εισόδου για ένα μόνο βήμα, δηλαδή σπάσιμο των αρχικών γραφών σε 2 κομμάτια. Επίσης θέλει τις τιμές των μετρικών $ncuts$ για κάθε διχοτομηση. Εδώ δεν καταλαβα εάν ήθελε την αναδρομική $ncuts$, η οποία μέσα της εκτελεί την μη αναδρομική στην αρχή και μετά ελέγχει αν θα την ξαναχρησιμοποιήσει, και να βάλω τέτοια $T1, T2$ ώστε να τη χρησιμοποιήσει μόνο μια φορά και μετά να σταματήσει, που μάλλον αυτό εννοούσατε, ή να βάλω κατευθείαν την μη αναδρομική που απλά σπάει 1 φορά το γράφο σε 2 κομμάτια. Δεν μπορούσα να βρω τι $T1$ και $T2$ θέλει για να σπάει μόνο μια φορά οπότε έκανα το 2ο.

Παρακατω φαινονται τα αποτελεσματα:

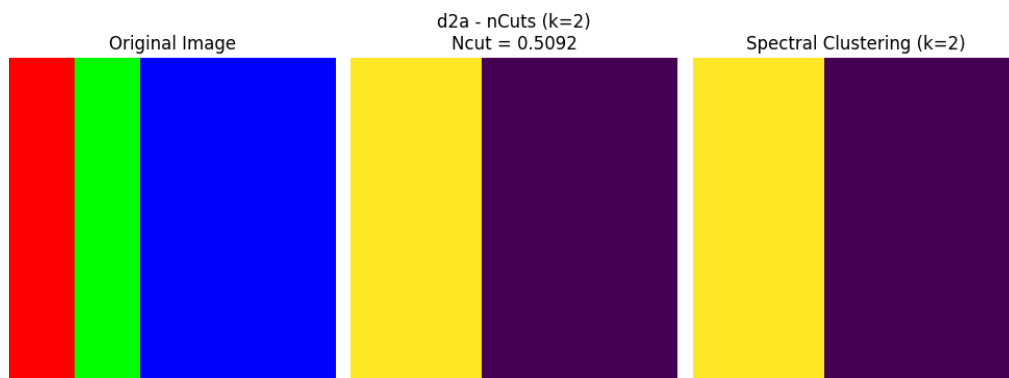


Figure 14: Εικόνα d2a - nCuts για $k=2$, $Ncut_Value$

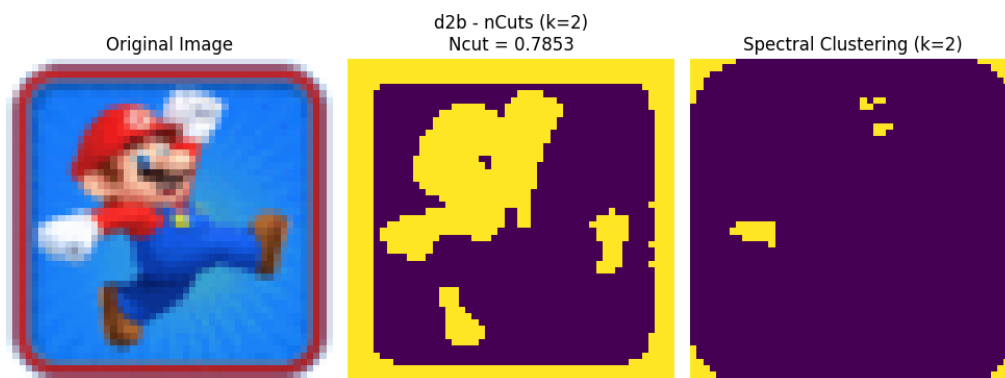


Figure 15: Εικόνα d2b - nCuts για $k=2$, Ncut_Value

9 demo3c.py

Τελος, εδω ζηταιει την ολοκληρωμενη αναδρομικη nCuts για τιμες των παραμετρων $T_1 = 5$ και $T_2 = 0.2$ για καθε εικονα εισοδου.

Παρακατω φαινονται τα αποτελεσματα:

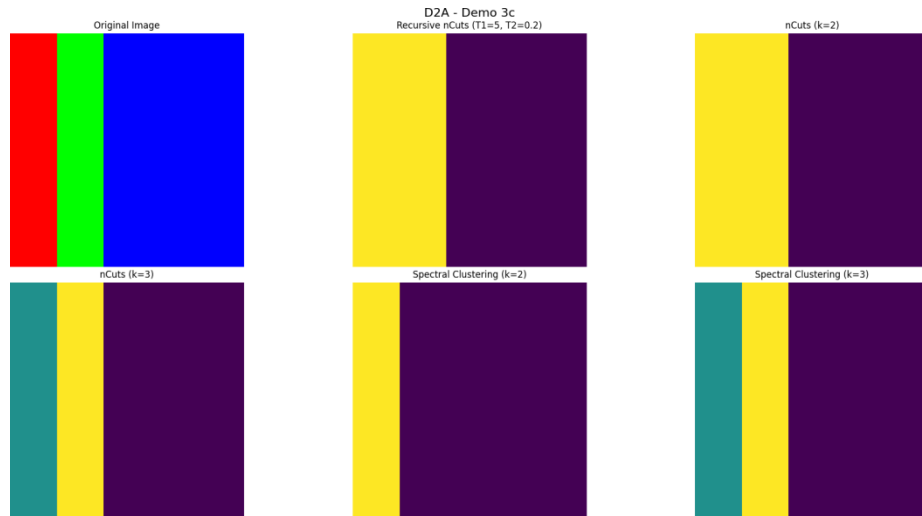


Figure 16: Εικόνα d2a - Original, recursive $T_1=5$, $T_2=0.2$, ncuts $k=2,3$, spectral $k=2,3$

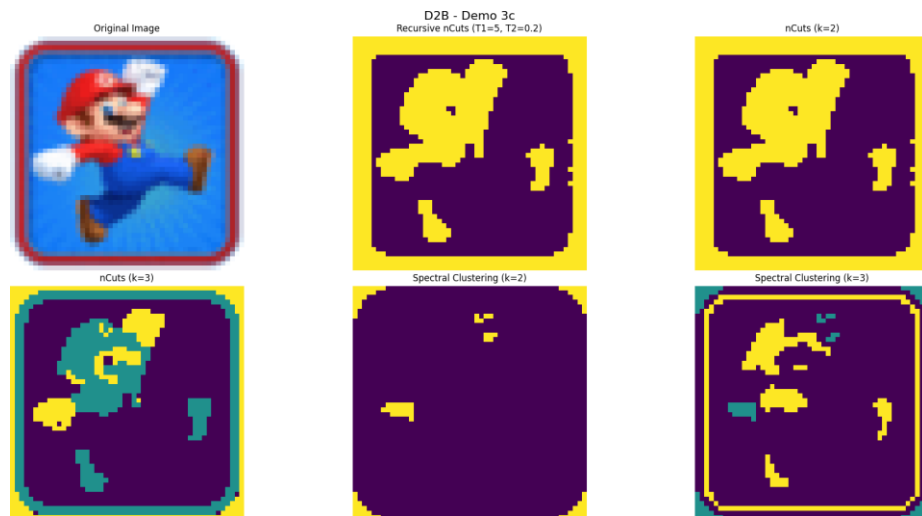


Figure 17: Εικόνα d2b - Original image, recursive ncuts $T_1=5$ $T_2=0.2$, ncuts $k=2,3$, spectral $k=2,3$

Παρατηρούμε ότι για τις δοσμένες τιμές T_1, T_2 , η recursive ncuts δίνει την ίδια εξοδό που δίνει και η non-recursive ncuts για $k = 2$. Αυτό συμβαίνει επειδή τα κατωφλία T_1, T_2 είναι τέτοια, που η recursive, αφού καλεστεί για πρώτη φορά την ncuts και σπασει τον αρχικό γραφο στα 2, μετα σταματάει.

Για εκπαιδευτικούς σκοπούς, επαιζα με τις τιμες T_1 και T_2 και εβαλα να μας δειχνει στο terminal τα σπασιματα που κανει. Αφηνω εδω αλλα 2 αποτελεσματα για τιμες κατωφλιων $T_1 = 5$ και $T_2 = 0.9$.

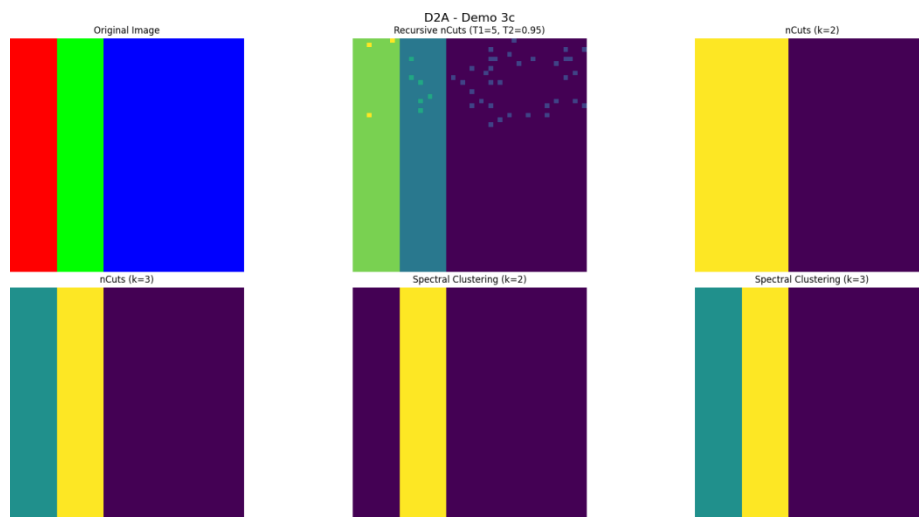


Figure 18: Εικονα d2a - Original, recursive $T_1=5$, $T_2=0.9$, ncuts $k=2,3$, spectral $k=2,3$

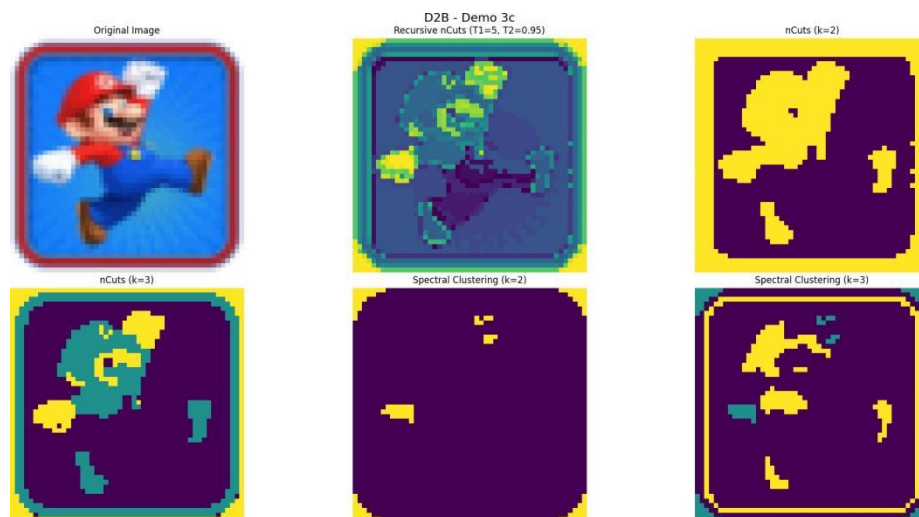


Figure 19: Εικονα d2b - Original image, recursive ncuts $T_1=5$ $T_2=0.9$, ncuts $k=2,3$, spectral $k=2,3$

Βλεπουμε οτι η αναδρομικη ncuts κανει την καλυτερη κατατμηση εικονας απο ολες τις μεθοδους, γιατι δημιουργει και ομαδοποιει clusters με μεγαλυτερη ακριβεια και λεπτομερεια, (οπως ειπαμε παραπανω, περισσοτερα και καλυτερα σπασιματα κλπ).

Τελικα ισως και να εγραψα πολλα, αλλα φταιει που πορωθηκα και γουσταρα την εργασια. Καλη συνεχεια καλο καλοκαιρι, μπανια να κανετε πολλα, μεχρι να σιχαθειτε τη θαλασσα. Εγω στη Ροδο ετσι θα κανω.

Φιλικους χαιρετισμους,
Ρουσος Σταματιος ΑΕΜ: 10704

Τελος Εργασιας